

# DBMS Project Handbook

*Fifteen full-stack projects for the Database Management System course. Each has a front-end, a back-end, and a database — the database is the graded core. Choose one project per group.*

## What every project must include

- **ER diagram** — entities, attributes (mark key, composite, multivalued, derived), relationships with cardinality and participation, and a weak entity where it fits.
- **Relational schema (3NF)** — the ER diagram converted to tables, primary keys underlined, foreign keys marked, normalized to third normal form; state the functional dependencies used.
- **A working database** — 6 to 8 related tables with proper primary/foreign keys and referential-integrity constraints.
- **SQL** — table creation (DDL), data operations (DML), and at least five meaningful queries using joins, GROUP BY and aggregate functions.
- **Login and roles** — at least two roles (Admin and User) with different access.
- **CRUD + one report** — create/read/update/delete on the main entities, and one summary screen built from aggregate queries.

## Getting your data

Design the schema first, then fill it. Do not hunt for a dataset that matches your tables — design your 3NF tables, create them, then populate them one of three ways:

- **Generate realistic data** (recommended) — Mockaroo ([mockaroo.com](https://mockaroo.com), up to 1,000 rows free, exports SQL/CSV) or ToolShelf ([toolshelf.tech/mock-data-generator](https://toolshelf.tech/mock-data-generator), more rows, browser-based).
- **Collect a little real data** from your own campus for the master tables (your actual subjects, the real canteen menu, one library shelf) — best for a convincing demo.
- **Public datasets** for reference only — Kaggle ([kaggle.com/datasets](https://kaggle.com/datasets)), data.gov.in (Indian open data), UCI ([archive.ics.uci.edu](https://archive.ics.uci.edu)).

**Study a real schema:** import a famous sample database and copy its style — Sakila (borrow/return), Chinook (orders and billing), classicmodels (orders + inventory + staff), employees (HR/payroll).

## Core technology stack

**Recommended (easiest):** HTML + CSS + JavaScript (with Bootstrap) for the front-end, PHP for the back-end, and MySQL for the database — all bundled in XAMPP, which includes phpMyAdmin for building tables visually.

**Front-end:** [w3schools.com/html](https://w3schools.com/html), [developer.mozilla.org](https://developer.mozilla.org), Bootstrap ([getbootstrap.com](https://getbootstrap.com)). **Back-end (pick one):** PHP ([php.net](https://php.net)), Python+Flask ([flask.palletsprojects.com](https://flask.palletsprojects.com)), Node+Express ([nodejs.org](https://nodejs.org)). **Database:** MySQL ([dev.mysql.com](https://dev.mysql.com)) or SQLite ([sqlite.org](https://sqlite.org), editor: [sqlitebrowser.org](https://sqlitebrowser.org)). **Tools:** XAMPP ([apachefriends.org](https://apachefriends.org)), VS Code ([code.visualstudio.com](https://code.visualstudio.com)), ER diagrams ([dbdiagram.io](https://dbdiagram.io) / [draw.io](https://draw.io)), charts ([chartjs.org](https://chartjs.org)).

**Use a relational database, not NoSQL.** MongoDB has no tables, keys or normalization, so it will not demonstrate what this course grades — use MySQL or SQLite.

## The 15 Projects

# 1. Hostel Management System

**Roles:** Student, Warden/Admin

**Core tables:** Hostel, Room, Bed, Student, AllotmentRequest, Allotment, Complaint, Notification

**Database:** MySQL

**Data:** Provided by the instructor (live occupancy + requests)

**Special feature:** Approval workflow + live occupancy

The Hostel Management System tracks every hostel, its rooms and beds, the students living in them, and the process of allotting a bed to a student. Its defining feature is that it runs on live occupancy data — at any moment the system knows how many beds are free in each room — and that a student cannot simply take a room. When a student requests a room or a room change, the request is stored as **pending** and waits for the warden to approve or reject it. Only on approval does the occupancy update and the student receive an alert.

In the database, the structure is a chain of one-to-many relationships — one hostel has many rooms, one room has many beds — with Student as a separate entity and an AllotmentRequest table that carries a status (pending / approved / rejected) along with who raised it, who approved it, and when. Because a student may occupy different beds over different terms, the allotment history is a many-to-many relationship between students and beds resolved by an Allotment table. Occupancy is a maintained count derived from current allotments, and referential integrity ensures a request can never point to a room that does not exist.

Build the approval workflow with a simple status column that you update as the request moves through the pipeline, and use login sessions to separate the student and warden roles so that only the warden sees the approve/reject screen. The reports — occupancy per hostel, list of pending requests, students with dues — are straightforward GROUP BY queries.

# 2. Timetable Management System

**Roles:** Admin, Professor

**Core tables:** Professor, Course, ClassGroup, Room, TimeSlot, Day, Timetable

**Database:** MySQL

**Data:** Provided by the instructor (professors, courses, slots)

**Special feature:** Clash detection

The Timetable Management System builds weekly timetables by assigning a professor to a course in a specific room and time slot, for a specific class group. Its central challenge is clash detection: the same professor cannot be in two rooms at once, a room cannot host two classes in the same slot, and a class group cannot have two lectures at the same time. The system must refuse to create such a clash.

This project is a clean illustration of an **n-ary relationship**. The core Timetable table combines Professor, Course, Room and TimeSlot, so it becomes its own table holding a foreign key to each of them — exactly the ER-to-relational rule for relationships of degree greater than two — with its primary key formed from the combination. Clash rules become uniqueness checks: no two timetable rows may share the same (Room, TimeSlot), the same (Professor, TimeSlot), or the same (ClassGroup, TimeSlot).

For the application, before inserting an assignment run an SQL EXISTS check for each of the three clash conditions and block the insert if any is found. Display the result as an HTML grid (days across, slots

down) rendered from the database, and offer per-professor and per-class views plus a free-room finder query.

### 3. Library Management System

**Roles:** Librarian, Member

**Core tables:** Book, BookCopy, Author, Category, Member, Issue, Return, Fine, Reservation

**Database:** MySQL

**Data:** Real books off one shelf; generate members & issues in Mockaroo

**Special feature:** Due-date & fine calculation

The Library Management System manages books, the physical copies of each book, members, and the borrowing between them, including fines for late returns. A member borrows a copy, the system records the issue and due dates, and on return it calculates any fine automatically from the number of days overdue.

The heart of the design is the **many-to-many relationship** between members and books, resolved by an Issue table that carries the issue date, due date and fine — a relationship with its own attributes. A single title can have several physical copies, so BookCopy behaves like a dependent of Book. The design naturally reaches third normal form once author and category details are pulled into their own tables rather than repeated on every book.

Fine calculation is simple date arithmetic in SQL or PHP. To model your schema, import and study the Sakila sample database ([dev.mysql.com/doc/sakila](http://dev.mysql.com/doc/sakila)), which handles exactly this rent-and-return pattern, or look at the open-source Koha library system for feature ideas.

### 4. Hospital / Clinic Management System

**Roles:** Receptionist/Admin, Doctor, Patient

**Core tables:** Doctor, Patient, Dependant, Department, Appointment, Prescription, Bill

**Database:** MySQL

**Data:** Generate in Mockaroo; real department names

**Special feature:** Appointment scheduling

The Hospital Management System records doctors, patients, and the appointments between them, along with prescriptions and billing. A patient books an appointment with a doctor for a date and time; the doctor records a prescription for that visit; and the visit generates a bill.

This project is the textbook home of the **weak entity**: a patient's dependant is stored only by name and relationship and cannot be identified without knowing which patient it belongs to, so its key is composite (patient key plus the dependant name). The doctor–patient link is a many-to-many relationship resolved by the Appointment table, which carries the date and fee as relationship attributes. Pulling department details into their own table keeps the design in third normal form.

For the application, handle appointment slots with date/time fields and a simple check that a doctor is not double-booked. Study the open-source OpenMRS medical-records project for domain structure, and the hospital directory on [data.gov.in](http://data.gov.in) for realistic department and specialisation names.

## 5. Online Examination System

**Roles:** Admin/Teacher, Student

**Core tables:** Exam, Question, Option, Student, Attempt, Answer, Result

**Database:** MySQL

**Data:** Type a real question bank; generate students & attempts

**Special feature:** Timer + auto-grading

The Online Examination System lets a teacher create multiple-choice tests and have them graded automatically. A student takes a timed exam, the system records each chosen answer, and it scores the attempt the moment it is submitted.

The design has two natural **weak entities**: an Option belongs to its Question and an Answer belongs to its Attempt — neither makes sense on its own. The student–exam link is many-to-many, resolved by the Attempt table, and results are computed with heavy use of **aggregate queries** (score per attempt, average per question, pass percentage). Keeping the correct-answer flag on the Option, and the chosen option on the Answer, keeps the design clean and normalized.

The timer is a JavaScript countdown (setInterval) that auto-submits at zero, and auto-grading is a query that compares each Answer against the correct Option. Randomising question order is an easy enhancement.

## 6. Online Shopping (E-commerce) System

**Roles:** Admin, Customer

**Core tables:** Customer, Product, Category, Cart, CartItem, Order, OrderItem, Payment

**Database:** MySQL

**Data:** Generate products/customers/orders in Mockaroo

**Special feature:** Cart + order transaction

The Online Shopping System is a small store where customers browse products, add them to a cart, and place an order that is then paid for. The admin manages the product catalogue and views sales.

This is the classic **many-to-many with attributes** case: an order contains many products and a product appears in many orders, resolved by the OrderItem table, which stores the quantity and the price at the time of purchase. Categories live in their own table so product rows are not repeated, and placing an order — creating the Order, its OrderItems, and reducing stock — should be treated as a single transaction so the database is never left half-updated.

The cart is handled with sessions or browser localStorage before the order is committed. To model the schema, study the Chinook or classicmodels sample databases, whose invoice-and-line-item structure is exactly this pattern; Kaggle's Olist e-commerce dataset gives realistic product and order values.

## 7. Employee Payroll Management System

**Roles:** HR/Admin, Employee

**Core tables:** Employee, Department, Designation, Attendance, SalaryStructure, Payslip, LeaveRequest

**Database:** MySQL

**Data:** Generate employees/attendance in Mockaroo

**Special feature:** Salary generation + leave workflow

The Employee Payroll System manages employees, their departments and designations, attendance, and the generation of a monthly payslip from that attendance and a salary structure. It also handles leave requests through an approval workflow.

This project is good practice for spotting and removing a **transitive dependency**: attributes such as the department head depend on the department, not directly on the employee, and must be moved into a Department table to reach third normal form. The employee–department relationship is one-to-many, and monthly payroll totals come from aggregate queries over attendance and salary.

Payslip generation multiplies attended days by the salary structure; producing a downloadable PDF payslip (with a library such as jsPDF) is a strong optional extra. To model the schema, study the large MySQL employees sample database, which contains departments, titles and salaries over time.

## 8. Student Attendance Management System

**Roles:** Admin, Faculty, Student

**Core tables:** Student, Subject, Faculty, Enrollment, ClassSession, Attendance

**Database:** MySQL / SQLite

**Data:** Use your real class — actual students & subjects

**Special feature:** Percentage + shortage alerts

The Student Attendance System records attendance for each class session across subjects and reports each student's attendance percentage, flagging anyone who falls below the required threshold (typically 75%).

The student–subject relationship is **many-to-many**, resolved by an Enrollment table; each ClassSession belongs to a subject, and the Attendance table is a fact table keyed by student and session recording present or absent. Percentages are pure **GROUP BY aggregate queries** — attended sessions over total sessions — which makes this project a strong exercise in SQL reporting even though its schema is modest.

This is the one project where using real data is easiest and most convincing: enter your own class's actual students, subjects and sessions. A shortage report and a simple bar chart (Chart.js) of percentages per subject complete it.

## 9. Bus Reservation System

**Roles:** Admin, Passenger

**Core tables:** Bus, Route, Stop, Schedule, Seat, Passenger, Booking, Payment

**Database:** MySQL

**Data:** Generate routes/bookings in Mockaroo; real city names

**Special feature:** Seat locking transaction

The Bus Reservation System lets passengers search scheduled routes by source and destination, see which seats are free on a given bus, and book them — without any risk of two passengers booking the same seat.

A booking links a schedule, a seat and a passenger, which is a **ternary relationship** resolved by the Booking table. A route is an ordered sequence of stops (a one-to-many relationship with an ordering attribute), and preventing double-booking of a seat on a schedule requires a database **transaction** so that checking availability and reserving the seat happen atomically.

The front-end shows a seat-map grid coloured by availability. Study IRCTC or RedBus as a domain model for the route–schedule–seat–booking structure, and read the MySQL transaction documentation (COMMIT / ROLLBACK) for the seat-locking logic.

## 10. Blood Bank Management System

**Roles:** Admin/Staff, Donor, Requester

**Core tables:** Donor, BloodGroup, BloodUnit, Donation, Recipient, Request, Hospital

**Database:** MySQL

**Data:** Generate donors/stock; realistic group distribution

**Special feature:** Group-wise stock + request approval

The Blood Bank Management System tracks blood stock by group, registers donations from donors, and processes requests from recipients or hospitals through a staff-approved workflow, alerting when a group runs low.

Each BloodUnit comes from a Donor (a one-to-many relationship) and belongs to a blood group; group-wise stock is an **aggregate query** over available units. A Request carries a status and is approved by a staff member before a unit is issued — the same approval-workflow pattern as the hostel system, with status, approver and timestamp columns — and referential integrity ties every unit back to a real donor.

The alerting feature is a query that lists groups whose available count falls below a threshold. For realistic domain structure, study the national e-RaktKosh portal, and the blood-bank directory available on data.gov.in.

## 11. Inventory / Stock Management System

**Roles:** Admin, Store Manager

**Core tables:** Product, Category, Supplier, Purchase, PurchaseItem, Sale, SaleItem, Stock

**Database:** MySQL

**Data:** Generate products/suppliers/movements in Mockaroo

**Special feature:** Reorder alerts + charts

The Inventory Management System tracks products, their suppliers, and the movement of stock in (purchases) and out (sales) for a shop or warehouse, alerting when a product falls below its reorder level.

This project contains **two separate many-to-many relationships** — Purchase-to-Product and Sale-to-Product — each resolved by its own line-item table carrying quantity and price. Current stock is a maintained quantity updated by every purchase and sale, and supplier details sit in their own table to avoid repetition, keeping the design normalized.

The reorder alert is a query listing products whose stock is below their threshold; supplier-wise purchase summaries and a stock chart (Chart.js) make the report screen. The Northwind and classicmodels sample databases are excellent structural references.

## 12. Event Management System

**Roles:** Organizer/Admin, Participant

**Core tables:** Event, Venue, Organizer, Participant, Registration, Team, Result

**Database:** MySQL

**Data:** Use your real college fest events; generate registrations

**Special feature:** Registration + venue clash check

The Event Management System organises college events, assigns them to venues and time slots, and manages participant registrations — individual or team — with winners recorded at the end.

The participant–event relationship is **many-to-many**, resolved by a Registration table; a Team is a group entity that participants belong to; and, as in the timetable project, no two events may be scheduled in the same venue at the same time, which is a clash-detection query. Pulling venue and organizer details into their own tables keeps the design in third normal form.

This is another project where real data is easy to gather — use your own college fest's actual events and venues. A results screen and a participation-count report complete it.

## 13. Training & Placement Cell System

**Roles:** Placement Officer/Admin, Student, Company

**Core tables:** Student, Company, JobDrive, Application, Skill, StudentSkill, Offer

**Database:** MySQL

**Data:** Generate students/companies/drives; real CGPA ranges

**Special feature:** Eligibility filter + resume upload

The Training & Placement System links students and companies through the campus recruitment process. Companies post job drives, students apply, the system filters applicants by eligibility (CGPA, branch), and offers are tracked to completion.

This project is rich in **many-to-many relationships**: students to skills (via StudentSkill), students to drives (via Application), and students to offers. Eligibility becomes a set of constraint-based queries — selecting applicants whose CGPA and branch meet a drive's requirement — which is a strong SQL exercise, and normalizing skills into their own table avoids repeating them on every student.

The application handles resume file uploads (PHP file upload) and produces placement-statistics reports. Your own college's placement process, and sites like Internshala, are useful domain references.

## 14. Restaurant / Canteen Ordering & Billing System

**Roles:** Admin, Waiter/Cashier

**Core tables:** MenuItem, Category, TableSeat, Order, OrderItem, Bill, Payment, Chef

**Database:** MySQL / SQLite

**Data:** Use your real canteen menu & prices; generate orders

**Special feature:** Billing with tax + printable receipt

The Restaurant Ordering & Billing System takes food orders against tables, sends them to the kitchen, and generates an itemised bill with tax for each table.

The order–menu relationship is a **many-to-many with attributes** resolved by OrderItem (quantity per item), and the bill total is an **aggregate** over those line items plus tax. Menu categories live in their own table, and converting an order into a paid bill is a small transaction. The structure mirrors an invoice system closely.

Use your real canteen's menu and prices as the master data — it makes the demo convincing. The Chinook sample database's invoice-and-line structure is the ideal model, and a printable receipt is easily done with the browser's print function.

## 15. Gym / Fitness Centre Membership System

**Roles:** Admin, Trainer, Member

**Core tables:** Member, MembershipPlan, Trainer, Subscription, Attendance, Payment, WorkoutPlan

**Database:** MySQL / SQLite

**Data:** Generate members/plans/subscriptions in Mockaroo

**Special feature:** Expiry + renewal alerts

The Gym Membership System manages members, the plans they subscribe to, their trainers, attendance, and payments, and it alerts when a subscription is about to expire so it can be renewed.

A plan has many subscriptions (one-to-many), members and trainers meet in many-to-many training sessions, and the design offers clean **transitive-dependency** practice: a plan's fee depends on the plan, not on the member, and must live in the MembershipPlan table to stay in third normal form. Expiry checks are date comparisons against the subscription end date.

The renewal alert is a query listing subscriptions ending within, say, the next week; a revenue report aggregates payments by month. Any gym or fitness-app membership flow is a suitable domain reference.

### The minimum every group must show

- A **database** in MySQL (via phpMyAdmin) — 6 to 8 tables with primary and foreign keys, built from your 3NF design.
- A **front-end** — a few Bootstrap pages: login, one data-entry form, one list/report page.
- A **back-end** — scripts connecting the pages to the database (insert / display / update / delete).
- **SQL** — five meaningful queries (joins, GROUP BY, aggregates) demonstrated working.

A clean, well-normalized database with correct keys and a basic working application is already a complete project. The special feature listed for each project is what turns a good submission into an excellent one.